IoT-Based Smart Prepaid Energy Meter with Dual-Source Switching & Overcurrent Protection.

By Md Sohag Mahmud (BSc in EEE)

Project Abstract

Project Title: IoT Based Smart Prepaid Energy Meter with Dual-Source Switching & Overcurrent Protection.

- **Objective:** The primary goal of this project is to develop a modern, cost-effective, and IoT-based prepaid electricity meter. It will help consumers track their electricity consumption through a mobile app and prevent wastage through system load management.
- **Working Principle:** The main brain of this project is the NodeMCU (ESP8266). It measures the AC line current using the ACS712 current sensor to calculate the consumer's real-time power (Watt) and remaining balance. The data is simultaneously displayed on a local 1602 LCD display and transmitted via the internet to the Blynk IoT Cloud App.
- **Smart Features:**
    1. **Prepaid System:** When the balance reaches zero, the relay module automatically disconnects (turns OFF) the main AC line.
    2. **Dual-Source Auto Switching:** If any issue occurs in the AC source or if the balance runs out, it automatically shifts the load to a backup DC source (such as an IPS or battery).

Hardware Pin Connections

1. Controller and Display Block

Component	Component Pin	NodeMCU Pin	Note / Remarks
I2C LCD Display	VCC	**VU (or VIN)**	৫V পাওয়ার সাপ্লাই নিশ্চিত করতে
	GND	**GND**	কমন গ্রাউন্ডের সাথে যুক্ত
	SDA	**D2 (GPIO4)**	I2C ডাটা লাইন
	SCL	**D1 (GPIO5)**	I2C ক্লক লাইন

2. Logic and Control Block (Relay and Sensor)

Component	Component Pin	NodeMCU Pin	Note / Remarks
2-Channel Relay	VCC	VIN (or VU)	5V supply to drive the relay
	GND	GND	Must be common with the battery and NodeMCU
	IN1 (AC Relay)	D5 (GPIO14)	Active LOW signal logic
	IN2 (DC Relay)	D6 (GPIO12)	Active LOW signal logic
ACS712 Sensor	VCC	VIN (or VU)	5V input power
	GND	GND	Common ground
	OUT	A0 (Analog)	Connected through the midpoint of a voltage divider made of 10k and 20k resistors

---

3. Power and Load Block (AC/DC High Voltage)

- **AC Phase (Main Line):** Connects directly to the COM pin of Relay 1.
- **Relay 1 NO (Normally Open) Pin:** A wire runs from here into the 1st terminal of the ACS712 current sensor.
- **ACS712 2nd Terminal:** A wire runs from here to the Phase (Live) line of the AC LED bulb.
- **AC Neutral Line:** Connected directly to the neutral holder of the bulb.
- **DC Battery (+):** Connects to the COM pin of Relay 2, and the output goes to the positive terminal of the DC light (DC ground will be common).

Blynk Code:

```
#define BLYNK_TEMPLATE_ID   "TMPL6_CLaR165"
#define BLYNK_TEMPLATE_NAME "Smart Energy Meter"
#define BLYNK_AUTH_TOKEN    "RTJaF7jbiS18O1wtAgx8XexC2qEFSLzV"
#define BLYNK_PRINT Serial

#include <ESP8266WiFi.h>
#include <BlynkSimpleEsp8266.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
```

```
#define CURRENT_SENSOR_PIN A0
#define AC_RELAY_PIN       D3
#define DC_RELAY_PIN       D6
#define DC_SENSE_PIN       D7

LiquidCrystal_I2C lcd(0x27, 16, 2);

char ssid[] = "XXXX";
char pass[] = "XXXX";

float balance = 100.0;
float current_amps = 0.0;
float power_watts = 0.0;
const float vout_per_amp = 0.185;
unsigned long last_update = 0;
String current_source = "NONE";
String last_source = "NONE";

BLYNK_WRITE(V0) {
  float recharge_amount = param.asFloat();
  balance += recharge_amount;
  Blynk.virtualWrite(V1, balance);
}

void setup() {
  Serial.begin(115200);
  delay(500);

  pinMode(AC_RELAY_PIN, OUTPUT);
  pinMode(DC_RELAY_PIN, OUTPUT);
  pinMode(DC_SENSE_PIN, INPUT);

  digitalWrite(AC_RELAY_PIN, HIGH);
  digitalWrite(DC_RELAY_PIN, HIGH);

  lcd.init();
  lcd.backlight();
  lcd.setCursor(0, 0);
  lcd.print("Connecting WiFi");

  WiFi.persistent(false);
  WiFi.disconnect(true);
  delay(1000);
  WiFi.mode(WIFI_STA);
  delay(500);
  WiFi.begin(ssid, pass);
```

```
  int wifi_tries = 0;
  while (WiFi.status() != WL_CONNECTED && wifi_tries < 60) {
    delay(500);
    Serial.print("Status: ");
    Serial.println(WiFi.status());
    wifi_tries++;
  }

  if (WiFi.status() == WL_CONNECTED) {
    Serial.println("WiFi Connected!");
    Serial.println(WiFi.localIP());
    lcd.setCursor(0, 1);
    lcd.print("WiFi OK!        ");
    delay(500);

    Blynk.config(BLYNK_AUTH_TOKEN);
    Blynk.connect(5000);

    if (Blynk.connected()) {
      Serial.println("Blynk Connected!");
    } else {
      Serial.println("Blynk FAILED!");
    }

  } else {
    Serial.println("WiFi FAILED!");
    lcd.setCursor(0, 1);
    lcd.print("WiFi FAILED!    ");
    delay(1000);
  }

  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("Hybrid Energy");
  lcd.setCursor(0, 1);
  lcd.print("Meter Ready!    ");
  delay(1000);
  lcd.clear();
}

void switchToAC() {
  if (last_source != "AC MAIN") {
    Serial.println("Switching to AC...");
    digitalWrite(DC_RELAY_PIN, HIGH);
    delay(300);
```

```
    digitalWrite(AC_RELAY_PIN, LOW);
    last_source = "AC MAIN";
    Serial.println("AC ON");
  }
  current_source = "AC MAIN";
}

void switchToDC() {
  if (last_source != "SOLAR") {
    Serial.println("Switching to DC...");
    digitalWrite(AC_RELAY_PIN, HIGH);
    delay(300);
    digitalWrite(DC_RELAY_PIN, LOW);
    last_source = "SOLAR";
    Serial.println("DC ON");
  }
  current_source = "SOLAR";
}

void switchOff() {
  if (last_source != "CUT-OFF") {
    Serial.println("CUT-OFF!");
    digitalWrite(AC_RELAY_PIN, HIGH);
    digitalWrite(DC_RELAY_PIN, HIGH);
    last_source = "CUT-OFF";
  }
  current_source = "CUT-OFF";
}

void loop() {
  Blynk.run();

    if (!Blynk.connected() && WiFi.status() == WL_CONNECTED) {
    static unsigned long next_connect = 0;
    if (millis() > next_connect) {
      Serial.println("Reconnecting Blynk...");
      Blynk.connect(1000);
      next_connect = millis() + 15000;
    }
  }

  // ── Non-blocking current sampling ──
  static unsigned long sample_start = 0;
  static float current_sum = 0;
  static int read_count = 0;
```

```
  if (millis() - sample_start < 20) {
    int raw_analog = analogRead(CURRENT_SENSOR_PIN);
    float voltage = (raw_analog / 1024.0) * 3.3 * 1.515;
    float sample_current = (voltage - 2.5) / vout_per_amp;
    current_sum += (sample_current * sample_current);
    read_count++;
  } else if (read_count > 0) {
    current_amps = sqrt(current_sum / read_count);
    if (current_amps < 0.15) current_amps = 0.0;
    current_sum = 0;
    read_count = 0;
    sample_start = millis();
  } else {
    sample_start = millis();
  }

  if (millis() - last_update >= 1000) {
    int dc_available = digitalRead(DC_SENSE_PIN);

    if (dc_available == HIGH) {
      power_watts = current_amps * 12.0;
    } else {
      power_watts = current_amps * 220.0;
    }

    if (power_watts > 0 && balance > 0) {
      float cost_per_second = (power_watts / 1000.0) * (7.0 / 3600.0);
      balance -= cost_per_second;
      if (balance < 0) balance = 0;
    }

    // ── Relay switching ──
    if (dc_available == HIGH) {
      switchToDC();
    } else {
      if (balance > 0) {
        switchToAC();
      } else {
        switchOff();
      }
    }
    update_display();
    last_update = millis();
  }
}
```

```
void update_display() {
  lcd.setCursor(0, 0);
  lcd.print("Pwr:" + String(power_watts, 1) + "W " + current_source + "    ");
  lcd.setCursor(0, 1);
  lcd.print("Bal:" + String(balance, 2) + "Tk    ");

  if (Blynk.connected()) {
    Blynk.virtualWrite(V1, balance);
    Blynk.virtualWrite(V2, power_watts);
    Blynk.virtualWrite(V3, current_amps);
    Blynk.virtualWrite(V4, current_source);
  }
}
```

🚀 Future Scope

- ⚡ **Anti-Tampering System:** If someone attempts to open the meter cover illegally, an IR sensor or limit switch will automatically disconnect the power supply and send a theft alert to the electricity office.
- 📉 **Real-Time AC Voltage Monitoring (ZMPT101B Integration):** Currently, the voltage is assumed to be fixed at 220V. In the future, integrating a ZMPT101B voltage sensor will help protect home appliances from accidents caused by high or low voltage.
- 📱 **Tariff-Based Billing (Time-of-Use Tariff):** By adding a Real-Time Clock (RTC) module to the code, the system can automatically calculate different electricity rates for peak and off-peak hours.
- 🤖 **Machine Learning and AI (Load Forecasting):** By analyzing the consumer's daily electricity usage patterns, an AI-powered app can alert the user about how many days their current balance will last.

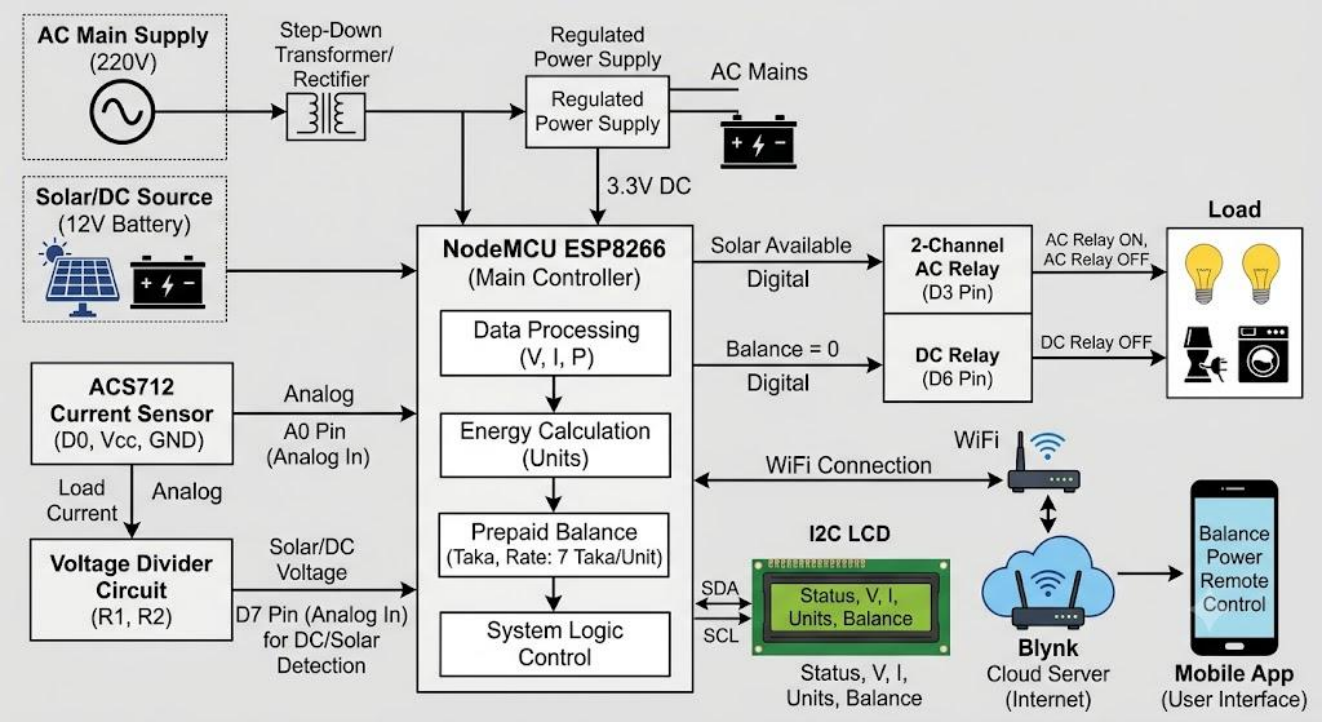

Fig: Block diagram

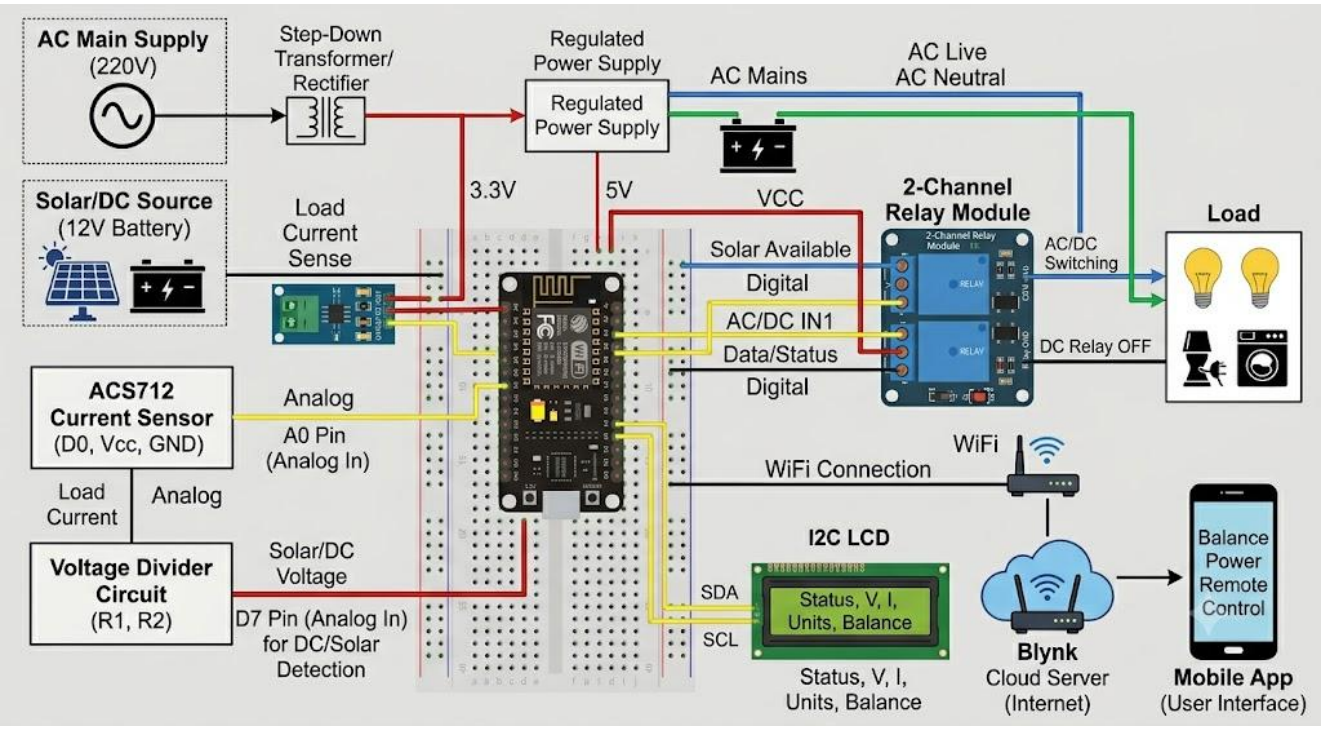

Fig: Connection diagram